

第 1 次课教师讲义：从 0 启动 LoongArch miniOS

技术编号：week01 | 检查点 tag：week01-qemu-hello | 约 2 节课
称“第 1 次课”，不称“第 1 周”。本课主题固定，作为整门课的工程入口。

0. 一句话目标

让学生亲眼看到：CPU 从 `_start` 进入 `kernel_main`，再经 `printk` / UART 打出 Hello。

1. 课程定位 (Why)

操作系统与系统软件不是从应用的 `main/printf` 开始的。在有 OS 的环境里，C 运行库和内核已经准备好栈、参数和标准输出；在 miniOS 里，这些要自己搭。

第 1 次课只打通**最小可运行闭环**，不追求一次讲完 ABI、异常、链接细节（后续课回填）：

CPU

- `boot/start.S` 中的 `_start`
- 设置 `$sp`
- `bl kernel_main`
- `printk` → `uart_putc`
- 写 UART MMIO
- 终端出现 Hello miniOS on LoongArch64

在 16 次课中的位置

课次	关系
第 1 次课	工程入口：会跑、会认路径
第 2 次课	补上全局变量可信状态
第 3–6 次课	回填寄存器、指令、程序设计、调用约定
第 7 次课起	构建、调试、服务、板级、综合

课堂话术：

今天先让机器“说话”；从第 3 次课开始，我们再把每条指令读透。

2. 教学目标

1. 能说出为何裸机/OS 相关代码常从汇编入口开始。
2. 能在工程中指出 `_start`、`kernel_main`、`printk`、`UART0_BASE`。
3. 能解释 `$sp` 为何要在进入 C 前设置。
4. 能区分 `bl`（调用）与 `b`（无条件跳转）的基本差异。
5. 能完成 `make` / `make run`，并记录**真实**串口输出。

验收输出（仅第 1 次课）：

Hello miniOS on LoongArch64

请使用 tag：week01-qemu-hello。

若当前 master 已含第 2 次课检查输出，课堂演示仍以本 tag 为准。

3. 课前准备

```
git fetch --tags
git switch --detach week01-qemu-hello
# 或: git switch -c demo-week01 week01-qemu-hello
make clean && make && make run
```

重点文件：

- boot/start.S
- kernel/main.c
- kernel/printk.c
- include/uart.h
- Makefile、kernel/linker.ld

4. 课程知识

4.1 为什么不能直接 printf

层级	谁提供
用户态 printf	C 库 + 系统调用 + 内核驱动
miniOS printk	自己遍历字符串 + 直接写串口寄存器

没有内核时：输出一个字符 $\approx$ 向固定地址写入一字节 (MMIO)。

4.2 工程认路 (5 分钟)

boot/start.S	入口、设栈、(第 2 次课起还有 clear_bss)
kernel/main.c	C 主函数
kernel/printk.c	输出实现
include/uart.h	串口基址
kernel/linker.ld	ENTRY(_start) 与段布局
Makefile	交叉编译与 QEMU

4.3 启动代码精读

汇编精讲：_start (Hello 路径)

说明：若当前检出的是含 clear_bss 的代码，课堂可先聚焦“设栈 + 调 main + halt”，clear_bss 留给第 2 次课。Hello tag 上的最小路径如下。

```
_start:
    la.global $sp, boot_stack_top /* 设置内核栈 */
    bl kernel_main /* 进入 C */
halt:
    idle 0
    b halt
```

逐条讲解：

1. `la.global $sp, boot_stack_top`

- `la.global` : Load Address, 把符号地址装入寄存器。
- 不是 “读内存里的数”, 而是 “得到 `boot_stack_top` 在哪”。
- `$sp` : 栈指针。栈从高地址向低地址增长, 故初始化指向栈顶高地址。
- 未设 `$sp` 就进 C, 一旦使用局部变量就可能踩未知内存。

2. `bl kernel_main`

- Branch and Link : 先把 “下一条指令地址” 写入 `$ra`, 再跳到 `kernel_main`。
- 这是 “汇编调用 C 函数” 的基本形态 (第 6 次课系统讲调用约定)。

3. `idle 0`

- 提示实现进入低功耗等待。
- `kernel_main` 若返回, CPU 不应 “掉出程序末尾”。

4. `b halt`

- 无条件跳转, 形成死循环, 保证停在可控状态。

```
.section .bss.stack, "aw", @nobits
.align 12
boot_stack:
.space 4096
boot_stack_top:
```

汇编精讲: 内核栈空间

1. `.section` : 把栈放到未初始化类段。
2. `.align 12` : 按 $(2^{12}=4096)$ 对齐。
3. `.space 4096` : 预留 4KiB; `boot_stack_top` 在高地址端。

4.4 C 主函数与 UART

```
void kernel_main(void)
{
    printk("Hello miniOS on LoongArch64\n");
    while (1) {
        __asm__ volatile("idle 0");
    }
}
```

```
volatile unsigned char *uart = (volatile unsigned char *)UART0_BASE;
*uart = (unsigned char)ch;
```

- `volatile` : 阻止编译器优化掉 “看似无用” 的 MMIO 写。
- `UART0_BASE` : QEMU virt 下常见串口直映地址 (见 `include/uart.h`)。
- 遇 `'\n'` 时补 `'\r'`, 适应终端 CRLF。

4.5 链接入口

```
ENTRY(_start)
. = 0x9000000000200000;
```

告诉链接器：程序从 `_start` 开始；装载窗口匹配 QEMU virt 常见布局（细节第 7 次课展开）。

5. 课堂 Demo (How)

建议顺序：

1. 展示目录（30 秒）
2. `make` 生成 `build/minios.elf`
3. `make run` 看 Hello
4. (可选) `objdump -d` 找 `_start`
5. (可选) 检查点：week01-00-skeleton → week01-03-printk-uart

tag	现象
week01-00-skeleton	无输出
week01-01-stack-setup	有栈，仍无输出
week01-02-kernel-main-empty	进 C，仍无输出
week01-03-printk-uart / week01-qemu-hello	Hello

6. 实验实践 (Do)

1. 环境检查：交叉 `gcc`、`make`、`qemu`
2. 阅读 `boot/start.S` 三问（见下）
3. 记录真实 `make run` 输出到报告

三问：

1. CPU 第一条指令在哪里？
2. 为什么不是从 `main` 开始？
3. 为什么不能直接 `printf`？

7. AI 共学

第 1 次课建议：解释与画图为主，不直接生成实验代码。

可问：解释 `b1/b`；画 Hello 数据通路；说明 QEMU 角色。

8. 思考与拓展

1. 删掉设 `$sp` 可能出现什么现象？如何用 GDB 验证？（第 8 次课）
2. `printk` 与 Linux `printk`、用户态 `printf` 各差在哪一层？
3. 为什么先 QEMU 再开发板？

9. 板书建议

```

_start
    la.global $sp, boot_stack_top
    bl kernel_main
kernel_main
    printk → uart_putc → *UART
halt: idle; b halt

```

附录：本课指令速查

指令	含义	课堂一句
la.global rd, sym	符号地址 → 寄存器	取地址，不是取值
bl target	调用	写 \$ra 再跳
b label	无条件跳	不保存返回地址
idle 0	等待	配合死循环 halt

寄存器	本课角色
\$sp	栈指针
\$ra	bl 写入的返回地址
\$zero	恒 0 (后续课常用)